

LLMs are context-limited

Giving AI the context it needs
with the Model Context Protocol

Andrea Manzini

SUSE QE-Workshop, June 2026

Today's agenda

1. **The Problem** : context-limited LLMs and the paste-the-logs era
2. **What is MCP?** : protocol, primitives, transport
3. **Building rpm-mcp** : a concrete server, end to end
4. **How Vector Search Works** : embeddings without the math
5. **Production Lessons** : pgvector, dedup, benchmarks
6. **Q&A**

~60 minutes. Please stop me with questions.

A simple question ?

"Claude, what security fixes landed in `vim` on openSUSE last month?"

A simple answer ?

"Claude, what security fixes landed in **vim** on openSUSE last month?"

Claude's answer:

"I don't have access to your distribution's package history or real-time data..."

The gap 🕒

LLMs know a lot. But they don't know **your stuff**.

- **Knowledge cutoff:** frozen at training time, months or years ago
- **No live data:** no access to your package repos, your servers, your logs
- **No query capability:** can't search, filter, or aggregate your data

The fix most people try: **paste things into the chat window**.

That doesn't scale.

The old way

You: Here is the changelog for vim. [pastes 500 lines]
What CVEs were fixed in the last 3 months?

Claude: [reads the paste]
Based on what you provided, CVE-2023-4738 was fixed in...

Problems:

- 128k, 256k context fills up fast
- Manual, not reproducible, not queryable
- New data? Paste again.
- No structured search, no date filters, no semantic queries

What if the LLM could just... ask?

```
Claude: [calls find_cve("CVE-2023-4738", "vim")]  
-> Found: vim 9.0.1847 (2023-09-04)  
    heap buffer overflow - bsc#1214924
```

```
Claude: The vulnerability was fixed in version 9.0.1847,  
    released September 4 2023. The patch addresses a  
    heap buffer overflow in vim_regsub_both()...
```

That's MCP.

What is MCP?

Model Context Protocol, standardized by Anthropic in November 2024.

"A standard protocol for giving LLMs access to tools and data."

Why it matters:

- Before MCP: every LLM product had its own plugin API, fragmented and non-portable
- After MCP: one protocol, any compatible client, any server you write

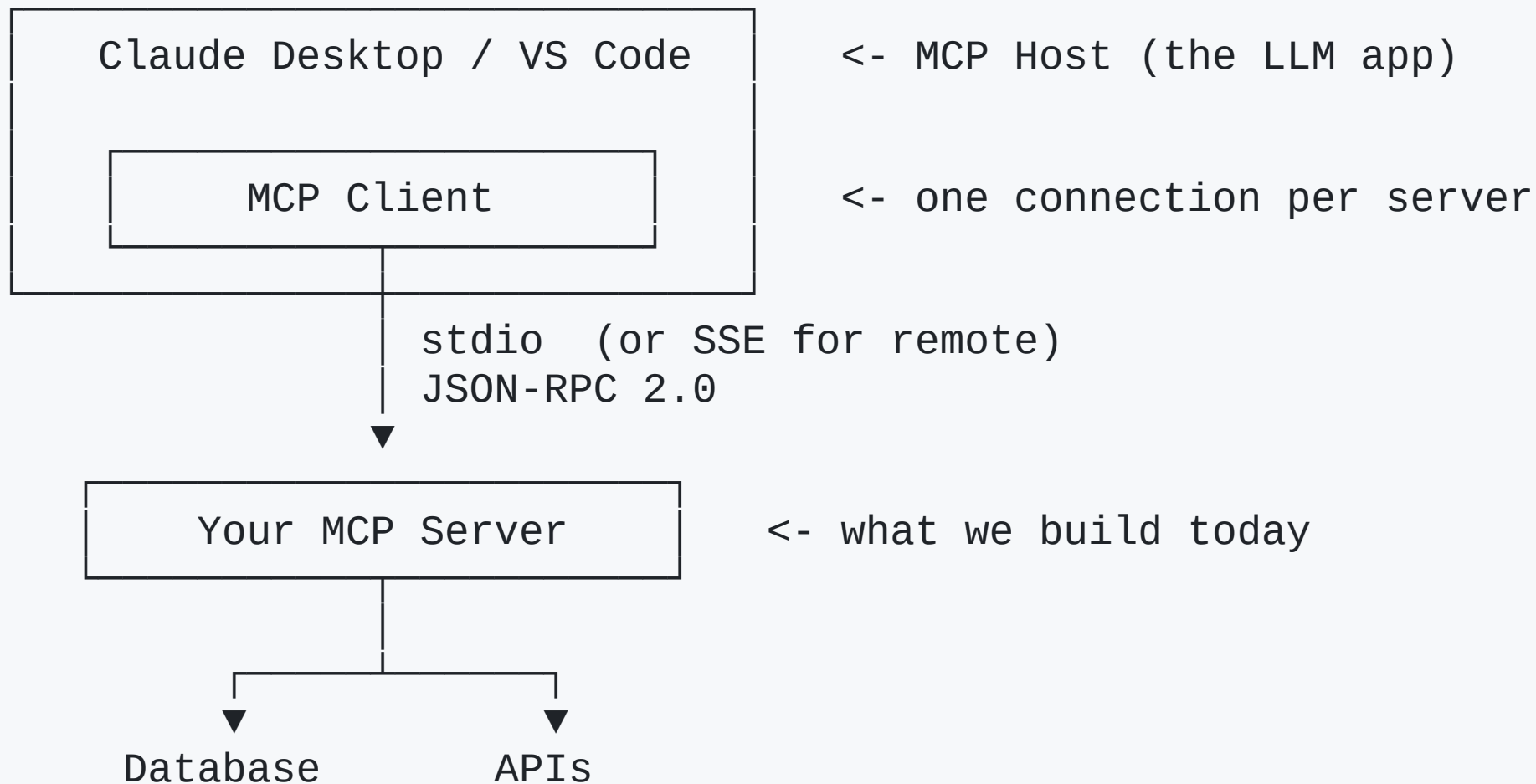
An MCP server is just a process that speaks JSON-RPC 2.0. That's it.

Before and after MCP

	Before MCP	After MCP
Protocol	Proprietary per vendor	JSON-RPC 2.0, open spec
Portability	Plugin works on one LLM	Works on any MCP client
Server	Vendor SDK required	Any language, any framework
Data access	Context paste or RAG hack	Native tool calls
Composability	None	Mix N servers per session

Claude, Gemini CLI, VS Code Copilot, LibreChat: all speak MCP.

Architecture: hosts, clients, servers 🏗️



Three primitives

Tools: functions the LLM can call

- Name, description (natural language), JSON Schema for inputs
- Return value goes back to the LLM as context
- Example: `find_cve(cve_id, package)` -> text with matching entries

Resources: data the LLM can read (like files)

- Identified by URI, read-only
- Less common in practice

Prompts: reusable prompt templates with parameters

- Predefined conversation starters
- Rarely used for data-access servers

Transport: stdio

Why stdio?

- **Zero config:** no ports, no TLS, no firewall rules
- **Zero auth:** the user's local account is the auth
- **Zero network:** pipe in/out of a subprocess
- **Process isolation:** each user gets their own server process

```
-> {"jsonrpc": "2.0", "id": 1, "method": "tools/call",  
    "params": {"name": "find_cve", "arguments": {"cve_id": "CVE-2023-4738"}}}  
  
<- {"jsonrpc": "2.0", "id": 1, "result": {"content": [{"type": "text",  
    "text": "Found 1 entry: vim 9.0.1847..."}]}}
```

You don't write this by hand. FastMCP handles it.

Hello world 🖐️

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-server")

@mcp.tool()
async def greet(name: str, times: int = 1) -> str:
    """Greet someone by name, optionally multiple times."""
    return "\n".join(f"Hello, {name}!" for _ in range(times))

mcp.run() # reads stdin, writes stdout
```

That's the complete server.

Hello world 🙌

- FastMCP infers **JSON Schema** from type hints (`str` , `int` , `bool` , `Optional`)
- The **docstring** becomes the tool description; the LLM reads it to decide when to call
- `mcp.run()` speaks the MCP wire protocol for you

How does the client discover tools? 🔍

Three-step handshake over JSON-RPC, once per session:

1. **initialize** : client and server exchange protocol version + supported primitives (**tools** , **resources** , **prompts**).
2. **tools/list** : client asks for the catalog. Server replies:

```
{
  "tools": [
    {
      "name": "find_cve",
      "description": "Case-insensitive search for a CVE ID...",
      "inputSchema": {
        "type": "object",
        "properties": {"cve_id": {"type": "string"}},
        "required": ["cve_id"]
      }
    }
  ]
}
```

3. Client injects the catalog into the LLM's system context. The **description** (your docstring) tells the LLM *when* to call. The **inputSchema** (from your type hints) tells it *how*.

Mid-session updates: server pushes `notifications/tools/list_changed` -> client re-fetches.

You write none of this. `@mcp.tool()` does it for you.

Connecting your server

Add to `~/.config/Claude/claude_desktop_config.json` (or the gemini-cli equivalent):

```
{
  "mcpServers": {
    "rpm-mcp": {
      "command": "uv",
      "args": [
        "run", "--directory", "/path/to/rpm-mcp",
        "mcp_server.py"
      ],
      "env": {
        "DATABASE_URL": "postgresql://rpm_mcp:rpm_mcp@localhost/rpm_mcp"
      }
    }
  }
}
```

You don't have to build from scratch

github.com/modelcontextprotocol/servers hosts dozens of official servers:

Server	What it gives the LLM
filesystem	Read/write files in a sandboxed path
github	Issues, PRs, code search
postgres	Run SELECTs against a database
sqlite	Same, for SQLite
slack	Read channels, post messages
puppeteer	Drive a browser
git	Local repo introspection
memory	Persistent knowledge graph

Plus hundreds of community servers in the [MCP registry](#).

Write your own only when nothing fits. rpm-mcp exists because no one had built a Linux-package server.

Our project: origin story

Pain: Claude/Gemini couldn't answer questions about [open]SUSE package history.

Experiment: How hard is it to build an MCP server that gives it access?

We had two existing side projects:

- `change log - poc` : openSUSE changelog ingestion / release notes with semantic search
- `rpm-spec-assistant` : Fedora/openSUSE spec parsing and analysis

Decision: merge them into one unified MCP server backed by PostgreSQL.

Result: 22 tools, 5 distros, 13k packages: `uv run mcp_server.py` .

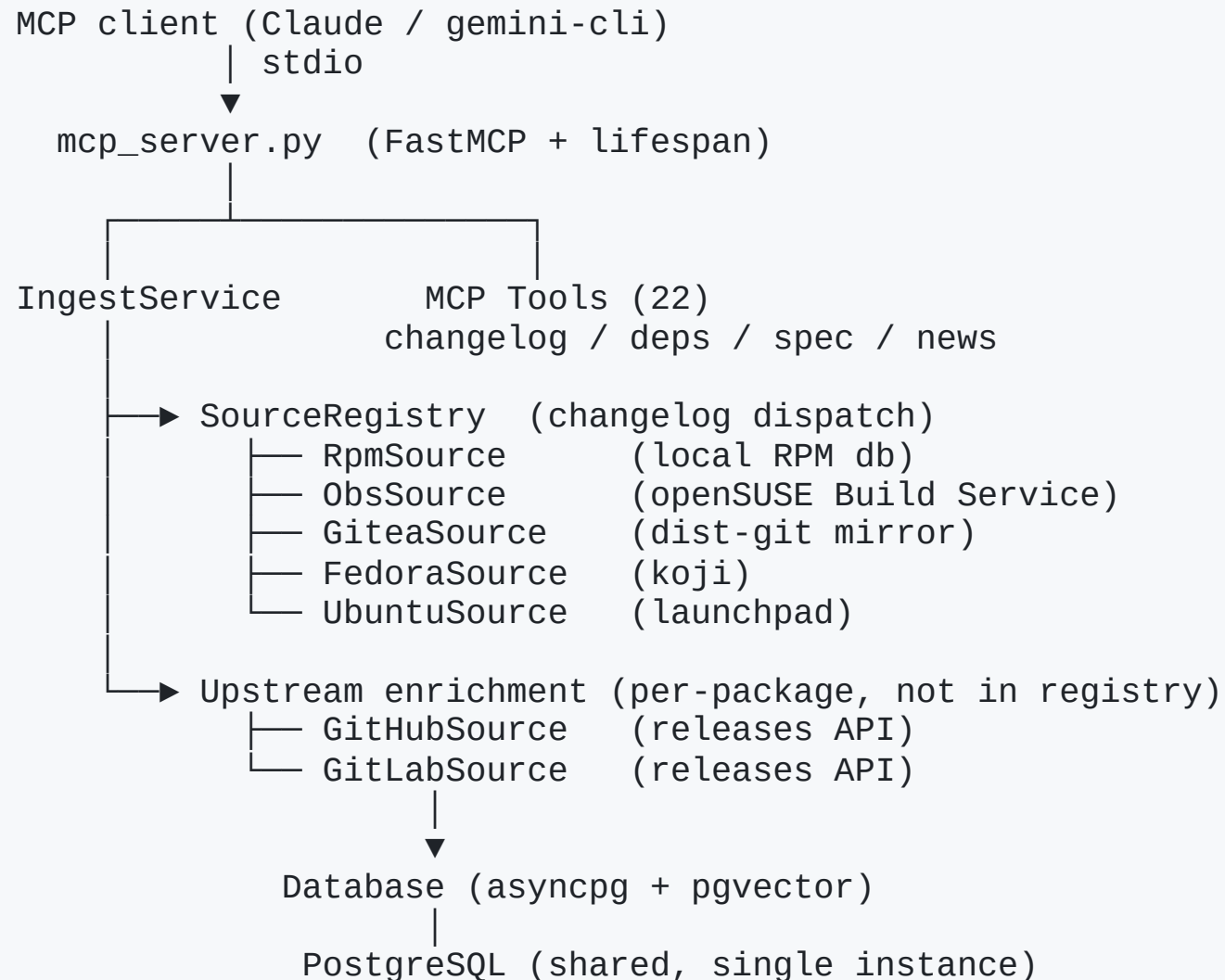
What we built

rpm-mcp: package intelligence for Linux distributions

- **22 MCP tools:** changelog search, CVE lookup, dependency graph, spec analysis, news
- **5 distros:** openSUSE, Fedora, Ubuntu, + GitHub/GitLab upstream releases
- **Scale target:** 13k packages, 100 concurrent users via shared Postgres
- **Single backing service:** PostgreSQL + pgvector (vectors + relational + FTS)
- **Local stdio model:** each user runs their own MCP server process

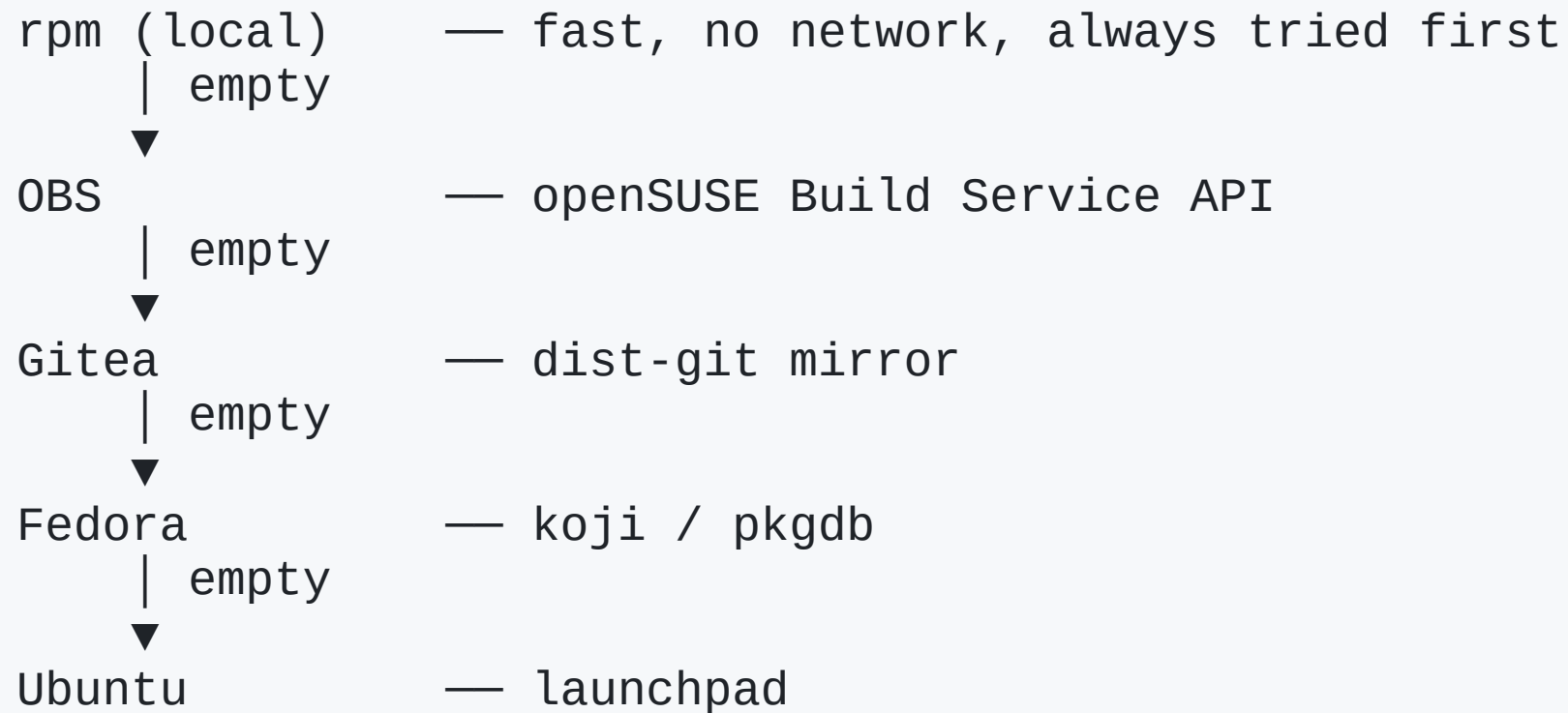
No Qdrant. No Redis. No separate FTS engine.

Architecture



Sources: the waterfall strategy

When fetching a package, try sources in order; **first non-empty wins**:

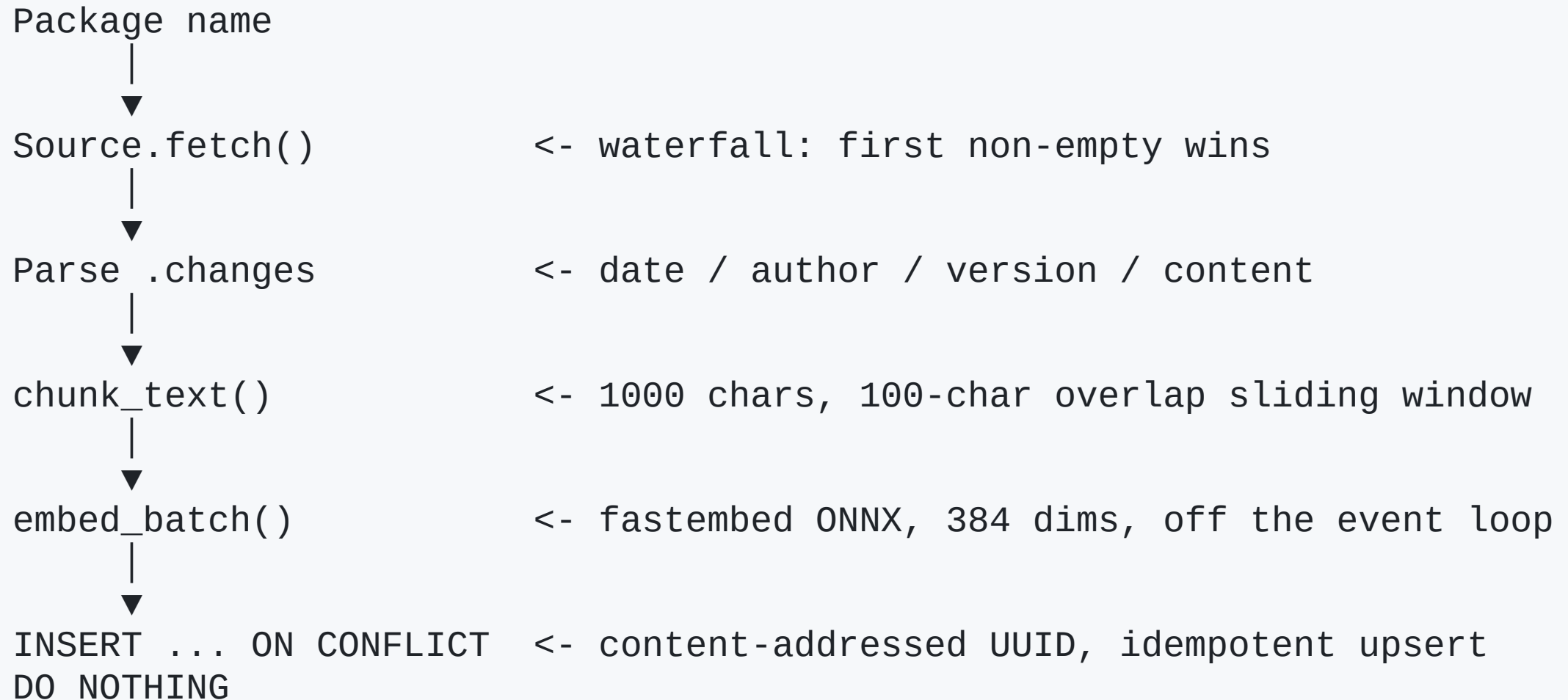


The registry filters by distro first, so an openSUSE package only hits openSUSE sources; a Ubuntu package only hits Ubuntu.

Parallel mode (for known-remote packages): local first, then all network sources concurrently; pick the one with the most entries.

Source failures are logged and skipped; stale cached data is served with a warning banner.

Ingestion pipeline ⚙️



Re-ingest the same package twice -> zero duplicates, zero extra work.

Tool surface

Category	Count	Representative tools
Changelog	12	<code>find_cve</code> , <code>semantic_search</code> , <code>get_recent_releases</code> , <code>analyze_package_diff</code>
Deps	4	<code>get_dependencies</code> , <code>get_reverse_dependencies</code> , <code>find_core_packages</code>
Spec	1	<code>get_spec_details</code>
News	5	<code>get_news</code> , <code>find_untested_changes</code> , <code>find_bugs_in_tests</code>
Total	22	

The LLM decides which tool to call based on the docstring. It reads them all at session start.

How are we going to find the stuff? 🤔

MCP solves **tool calling**. It does not solve **semantic matching** inside your data.

For `semantic_search`, we need to retrieve entries that mean the same thing, not just share keywords.

- Query: "openssl memory corruption fix"
- Relevant changelog: "CVE-2023-5363: heap buffer overflow in libssl"

Different words, same issue. Keyword search can miss this; embeddings make it retrievable.

What is an embedding? 🧠

Map text to a point in **high-dimensional space**. Nearby points = similar meaning.

```
"dog"      -> [0.82, 0.11, 0.73, 0.04, ...] (384 numbers)
"cat"      -> [0.79, 0.13, 0.71, 0.06, ...] <- close to "dog"
"airplane" -> [0.12, 0.88, 0.03, 0.91, ...] <- far from both
```

You don't choose what the dimensions mean. The model learns them from billions of text examples.

Key insight: similar *meaning* -> similar *coordinates*, even if the words are different.

The classic example

`king - man + woman ≈ queen`

In vector space, the "royalty" direction and "gender" direction are separable.

Why this matters for us:

`"heap buffer overflow in ssl library"`
≈
`"CVE-2023-5363: memory safety fix in openssl"`

Same meaning, completely different words. A keyword search finds nothing. A vector search finds both.

Cosine similarity

How do we measure "how close" two vectors are?

Cosine similarity: the cosine of the angle between two vectors.

```
cos(0°)    = 1.0    <- identical meaning  
cos(90°)   = 0.0    <- unrelated  
cos(180°)  = -1.0   <- opposite
```

Simple example (2D), both vectors already unit-length:

```
"security fix" = [0.80, 0.60]    mag =  $\sqrt{0.64+0.36}$  = 1.0  
"CVE patch"   = [0.60, 0.80]    mag =  $\sqrt{0.36+0.64}$  = 1.0  
  
dot product = (0.80×0.60) + (0.60×0.80) = 0.48 + 0.48 = 0.96  
similarity  = 0.96 / (1.0 × 1.0) = 0.96 <- very close (~16°)
```

In practice: 384 dimensions, not 2. But the math is the same.

Embeddings in practice

Model: BAAI/bge-small-en-v1.5 (fastembed, ONNX, runs locally, ~24MB)

```
async def semantic_search(query: str, limit: int = 5) -> str:
    """Natural-language search across indexed changelogs via pgvector."""
    emb = await embedder.embed_one(query) # -> list of 384 floats
    rows = await db.semantic_search(emb, limit=limit)
    ...
```

Query path:

1. Embed the user's query string -> 384-dim vector
2. Find the `limit` nearest stored vectors (cosine distance)
3. Return the matching changelog entries

Search latency (real benchmark, warm):

p50 = 8 ms | p95 = 12 ms | p99 = 12 ms

One DB to rule them all

Previous architecture (two side projects):

```
changelog-poc:      Qdrant (vectors) + SQLite (relational)
rpm-spec-assistant: SQLite only
```

Two services, two connection strings, two failure modes, two backup targets.

New architecture:

```
PostgreSQL + pgvector + pg_trgm
```

```
vectors          -> pgvector HNSW
full-text FTS     -> pg_trgm + tsvector
relational        -> plain SQL
transactions      -> ACID, always
```

One `DATABASE_URL`. One backup. One monitoring target. One `psql` to debug

pgvector: the numbers

HNSW index (Hierarchical Navigable Small World graph):

```
CREATE INDEX ON changelog_entries  
USING hnsw (embedding vector_cosine_ops);
```

HNSW builds a graph of approximate nearest neighbors; query traverses it instead of scanning all rows.

Scale: 650k vectors (13k packages x ~50 entries each)

Operation	p50	p95	p99
Semantic search (warm)	8 ms	13 ms	13 ms
Full ingest (network-bound)	2 s	11 s	11 s

Ingest is slow because it fetches from OBS/Gitea over the network. The DB itself is fast.

Content-addressed dedup

Same `.changes` block appears in OBS and the Gitea mirror -> same UUID -> no duplicate.

```
PKG_NAMESPACE = uuid.UUID("6ba7b810-9dad-11d1-80b4-00c04fd430c8")

def content_uuid(package: str, content: str) -> uuid.UUID:
    return uuid.uuid5(PKG_NAMESPACE, f"{package}::{content}")
```

Content-addressed dedup

```
INSERT INTO changelog_entries (id, package_id, content, embedding, ...)  
VALUES ($1, $2, $3, $4, ...)  
ON CONFLICT (id) DO NOTHING;
```

Properties:

- Deterministic: same input always -> same UUID
- Idempotent: re-ingest never creates duplicates
- No scanning: no `SELECT ... WHERE content = ?` before each insert
- Source-agnostic: OBS, Gitea, local RPM converge on the same row

Production reality

Deployment model: local stdio per user, one shared Postgres

```
User A: uv run mcp_server.py <-]
User B: uv run mcp_server.py <-]- all hit same PostgreSQL
User C: uv run mcp_server.py <-]
```

- No SSE, no auth, no TLS: all local
- Natural isolation: each user is a separate process
- Postgres handles concurrency; asyncpg pool per process

Production reality

Source failure handling:

```
WARNING: OBS fetch failed for 'vim'; serving cached data from 2026-05-30 12:34
```

Stale data is better than no data. The LLM sees the banner and reports it. No Prometheus, no Containerfile: deliberately out of scope for this deployment model.

Security & trust

The questions a SUSE audience always asks. Honest answers:

- **Tool selection is LLM-driven.** Don't expose `rm -rf` and trust the docstring to keep the LLM polite. If a tool is destructive, gate it behind explicit user confirmation in the client.
- **Stdio = same trust as the calling user.** No privilege escalation; the server runs as you.

Security & trust

- **Untrusted source data is sanitized at ingest.** `safe_upstream_url()` in `src/ingest.py:36` rejects `file://`, `http://internal/...`, etc. before we ever fetch.
- **No write tools in rpm-mcp.** Every tool is read-only over the cache. Worst case: the LLM gets stale data.
- **Prompt injection in changelog content?** Possible in theory : a hostile `.changes` entry could try to instruct the LLM. Mitigation: tool output is data, not instructions, and the client's system prompt says so.

Demo: semantic_search 🔍

Query: "TLS protocol weakness"

1. gnutls 3.8.13 (2026-04-29) - CVE-2026-33846 (CVSS high)
libgnutls: Add more checks to DTLS reassembly
2. curl 8.20.0 (2026-04-28) - CVE-2026-4873
Connection reuse ignores TLS requirement (bsc#1262631)
3. nginx 1.31.0 (2026-05-13)
HTTP/2 request injection via proxy_set_body directive

No keyword match between "TLS protocol weakness" and most of those entries ("DTLS reassembly", "connection reuse", "HTTP/2 injection"). The vector search found them by *meaning*.

Demo: find_cve

Query: `find_cve("CVE-2026-39881", "vim")`

Found 1 entry mentioning CVE-2026-39881:

Package: vim | Source: rpm

Version: 9.2.0398 | Date: 2026-04-25

Author: Martin Schreiner <martin.schreiner@suse.com>

- Fix bsc#1261833 / CVE-2026-39881.
- Update to 9.2.0398.
- Changes:
 - * 9.2.0398: MS-Windows: missing strtptime() support
 - * 9.2.0397: tabpanel: double-click opens a new tab
 - ...

Exact-match search over the cached changelogs. Sub-millisecond. No network call, no scraping, no paste.

Benchmark numbers

Real measurements against a local Postgres with ~10 packages ingested:

Operation	p50	p95	p99	notes
Semantic search	8 ms	13 ms	13 ms	warm, 10 queries
Full package ingest	2 s	11 s	11 s	network fetch included

Ingest breakdown:

- Fast packages (curl, vim, git): 300-400 ms (small changelog, fast OBS)
- Slow packages (chrony, nginx): 5-11 s (OBS rate-limiting, large history)

Search is always fast: vectors are in RAM via HNSW, query is CPU-only.

Test infrastructure

406 tests, 75% coverage

Three levels:

```
# Unit: fast, no containers, no network
./scripts/test.sh unit

# DB integration: real Postgres via testcontainers (Podman)
./scripts/test.sh e2e-db

# E2E: gemini-cli + real openSUSE packages
PYTHONPATH=. uv run pytest -m e2e
```

Key lessons:

- `TESTCONTAINERS_RYUK_DISABLED=true` : Ryuk can't mount the Podman socket
- Module-scoped async fixtures need `loop_scope="module"`

Common pitfalls when writing tools

Learned the hard way:

- **Returning 50 KB of text.** The LLM pays per token. Paginate, truncate, or summarize *at the tool level* : don't make the LLM do it.
- **Vague tool names.** `get_data(id)` will be picked at random. Use specifics: `get_package_change_log(name)` .
- **No type hints.** No schema -> LLM passes garbage -> tool crashes. Always type your parameters.

Common pitfalls when writing tools

- **Returning JSON-as-string.** The LLM has to re-parse it. Return plain text formatted for human reading; the LLM handles that natively.
- **Silent failures.** If a tool returns `""` on error, the LLM thinks it succeeded. Return `"ERROR: ..."` text.
- **Side effects in read tools.** Don't update state in a `get_*` tool : the LLM will call it multiple times.
- **Docstrings as afterthought.** The docstring IS the tool's contract with the LLM. Write it like an API description.

What MCP unlocks

Before:

- LLM knows the world, not your infra
- You paste logs; it answers about the paste

After:

- LLM calls your tools, gets live data, reasons on it
- Any compatible client (Claude, gemini-cli, VS Code, LibreChat)
- You own the data access layer; the LLM owns the reasoning

How to start:

1. `pip install "mcp[cli]"` (the official Anthropic SDK; ships `mcp.server.fastmcp`)
2. Write one tool that returns something useful
3. Add it to your MCP client config (see earlier slide)
4. Iterate

The protocol is stable. The tooling is production-ready.

Links and Q&A

- [This project:](#)

Specs:

- spec.modelcontextprotocol.io: full protocol spec
- github.com/anthropics/mcp: SDKs and examples

FastMCP:

- github.com/jlowin/fastmcp

pgvector:

- github.com/pgvector/pgvector

VISUAL doc : <https://ynarwal.github.io/how-llms-work/>

Grazie / Thank you!

LLMs are context-limited

Giving AI the context it needs with the Model Context Protocol

Andrea Manzini

SUSE QE-Workshop, June 2026